

Linux Programming: Assignment-8:

20-09-2025

NAME- G JYOTHSNA

ROLL NO- 04

USN- ENG24CY0021

CLASS- CYBER SECURITY

SECTION- C

11. What is a user-defined function in shell scripting? Explain with an example.

Ans:

A user defined function in shell scrpiting is a block of code that can be defined once and reused multiple times whenever it is called in script

It improves readability and avoids repetition of code in script

Example:

```
#!/bin/bash
```

```
greet_user() {
```

```
    echo "Hello, $1! Welcome to shell scripting."
```

```
}
```

```
greet_user "Jyothsna"
```

Output:

Hello, Jyothsna! Welcome to shell scripting.

Here greet_user() is a function

12. Write a bash script with a function that multiply two integer numbers.

Ans:

```
#!/bin/bash
multiply() {
    result=$(( $1 * $2 ))
    echo "The product of $1 and $2 is: $result"
}
read -p "Enter first number: " num1
read -p "Enter second number: " num2
multiply $num1 $num2
```

Output:

Enter first number: 5

Enter second number: 4

The product of 5 and 4 is: 20

13. Explain how arrays (1D, 2D, and 3D) are declared in bash scripting.

Ans:

In bash scripting, arrays are used to store multiple values in a single variable

1D Declaration:

```
arr=(10 20 30 40 50)
```

Accessing elements:

```
echo "First element: ${arr[0]}"
```

```
echo "All elements: ${arr[@]}"
```

Output:

First element: 10

All elements: 10 20 30 40 50

2D Declaration:

```
arr[0,0]=1
```

```
arr[0,1]=2
```

```
arr[1,0]=3
```

```
arr[1,1]=4
```

Accessing elements:

```
echo "Element at [0,0]: ${arr[0,0]}"
```

```
echo "Element at [1,1]: ${arr[1,1]}"
```

Output:

Element at [0,0]: 1

Element at [1,1]: 4

3D Declaration:

```
declare -A arr3D
```

```
arr3D["0,0,0"]=1
```

```
arr3D["0,0,1"]=2
```

```
arr3D["1,0,0"]=3
```

```
arr3D["1,1,1"]=4
```

Accessing element:

```
echo "Element at [1,1,1]: ${arr3D["1,1,1"]}"
```

Output:

Element at [1,1,1]: 4

14. Write a shell script to display elements of an array.

Ans:

```
#!/bin/bash
numbers=(10 20 30 40 50)
echo "Elements of the array are:"
# Using for loop
for num in "${numbers[@]}"
do
    echo "$num"
done
```

Output:

Elements of the array are:

10
20
30
40
50

15. What is the purpose of cron in Linux?

Ans:

Cron is a time-based job scheduler in linux

It uses a special configuration file called crontab(corn table) to define when and what commands should run

It is mainly used for:

Running backups automatically at night

Cleaning up temporary files regularly

Sending emails or reports on a schedule

Running system updates periodically

Cron in linux is used to automate scheduled tasks so user dont have to run them manually everytime

16. Write a cron job to run a backup script every day at midnight.

Ans:

```
0 0 * * * /home/user/backup.sh
```

This runs at midnight everyday

To make the script executable use commnd:

```
chmod +x /home/user/backup.sh
```

To schedule the corn job

```
crontab -l
```

17. How do you schedule a one-time job using at command?

Ans:

Syntax: at[TIME]

For example:

At 22:30

Use the prompt at> and run the command:

```
sh /path/to/backup.sh
```

Save it then the command will run at 10:30

To view the scheduled jobs run atq

Or just run the command:

```
echo "sh /path/to/script.sh" | at 08:00 tomorrow
```

18. Write a script to display disk usage using df and du.

Ans:

```
#!/bin/bash
```

```
echo "===== Disk Usage Summary (df) ====="
```

`df -h`

`echo ""`

`echo "==== Disk Usage of Current Directory (du) ===="`

`du -h --max-depth=1`

Give the execute permission before running it

19. How can you log the output of a script using the tee command?

Ans:

To run use:

`./your_script.sh | tee output.log`

To append use:

`./your_script.sh | tee -a output.log`

To log its output use:

`./disk_usage.sh | tee disk_usage.log`

20. Explain with an example how shell scripting can automate system administration tasks.

Ans:

Shell scripting is a powerful tool in linux that can automate repetitive system which can save alot time and reduce error and debugging

Example

`#!/bin/bash`

`SOURCE_DIR="/home/user/documents"`

`BACKUP_DIR="/home/user/backup"`

```
DATE=$(date +%Y-%m-%d-%H-%M)
tar -czf "$BACKUP_DIR/backup-$DATE.tar.gz" "$SOURCE_DIR"
echo "Backup of $SOURCE_DIR completed on $DATE"
>>"$BACKUP_DIR/backup.log"
```

This runs backup at 2:00 am daily

It saves alot of time reduces human error and ensures tasks run regularly without supervision it also makes system maintenance easier